Software Architecture Specification (SAS)

COS 301 - Capstone 2026
Capstone Project: EquityLens
By The Big Five

Project Name: EquityLens
Client: Amazon Web Services (AWS)
Team: The Big Five (TB5)

Team members:

- Abdulrahman Sabah (Team Lead) - u24566170
- Joshua Heath - u23541475
- Antony van Straten - u24590739
- Jonty Honey - u23536862

Contents

- 5.1 Deployment Requirements
- 5.2 Deployment Diagram (technology specific)
- 5.3 CI/CD Pipeline Diagram
- 5.4 Secrets Management
- 5.5 Rollback Strategy

1. Introduction

This document specifies how The Big Five (TB5) aimed to achieve the given specification provided by our client, Amazon Web Services.

2. Architectural Requirements

2.1 Architectural Patterns

2.1.1 Three-Tier Architecture

EquityLens follows a three-tier architecture:

- Presentation tier - a single-page web application responsible for rendering the UI and handling user interaction. It holds no business logic and never accesses the database directly.
- Application tier - an API backend containing all business logic: authentication orchestration, portfolio persistence and live pricing, indicator calculation, news correlation, AI prompt construction and guardrails.
- Data tier - a relational database storing user accounts, portfolio data, snapshots, watchlists, conversations, and cached market/news data.

The tiers communicate exclusively over HTTPS REST. This separation exists so each tier can be developed, tested, scaled and replaced independently.

2.1.2 Serverless Architecture

The application tier targets deployment as serverless functions behind a managed API gateway. Functions execute on demand, scale automatically with traffic, and incur no cost when idle unless implemented otherwise.

2.1.3 Layered Backend

Within the application tier the backend is layered:

- Routers handle HTTP concerns only: request parsing, response shaping, status codes, auth dependency injection.
- Services encapsulate business logic: portfolio aggregation and live pricing, indicator maths, news correlation, AI context building and guardrails.
- Repositories encapsulate data access: each aggregate (portfolios, holdings, transactions, users, watchlist, market data) has a repository owning its queries.

This means business logic is unit-testable without HTTP, and queries are changeable without touching business logic.

2.1.4 Client-Server with External Service Integration

Market data, news, identity and AI capabilities are consumed from external providers through dedicated service modules, each wrapped with caching and/or graceful degradation behaviour so that a provider outage degrades its own feature

2.2 Design Patterns

- Service Layer - backend/app/services/. Separates business logic from HTTP routing, services are unit-tested independently of the API layer.

- Repository - backend/app/repositories/. Each aggregate's data access is isolated behind a repository (e.g. PortfolioRepository, HoldingsRepository, UserRepository), so query changes never leak into business logic.
- Provider - frontend/src/context/. React context providers (AuthContext, PortfolioContext, ThemeContext) expose global state without prop drilling.
- Observer - React state/context updates. Component re-renders are driven by state changes - an implicit observer model across the UI.
- Factory - get_db() in app/database.py and get_bedrock_client() in the AI service. Consistent construction and lifecycle management of database sessions and external service clients.
- Facade - usePortfolio, useAuth and related hooks. Each hook presents a single simple interface over multi-step fetch/auth flows for UI components.
- Cache-Aside - app/utils/stock_cache.py and market_cache.py. Price history and fundamentals are looked up in the object-store cache first and fetched from the provider only on a miss, respecting rate limits.

2.3 Constraints

2.3.1 Financial Constraints

- **C-1:** All infrastructure must operate within AWS Free Tier limits at zero cost.
- **C-2:** All non-AWS tools must be open-source or free-tier.
- **C-3:** AI capabilities must use free-tier LLM APIs exclusively.

2.3.2 Security and Compliance Constraints

- **C-4:** All data must be encrypted in transit (TLS) and at rest (AES-256).
- **C-5:** All user data handling must be fully compliant with POPIA.
- **C-6:** Raw brokerage credentials must never be stored in the system.
- **C-7:** Portfolio data may only be imported via file upload or manual entry.
- **C-8:** User portfolio data must not be visible to any other user.

2.3.3 Integration Constraints

- **C-9:** Market data fetching must respect free-tier API rate limits, with caching to avoid redundant calls.
- **C-10:** The platform must support at least one South African brokerage for portfolio statement import.

2.3.4 AI Constraints

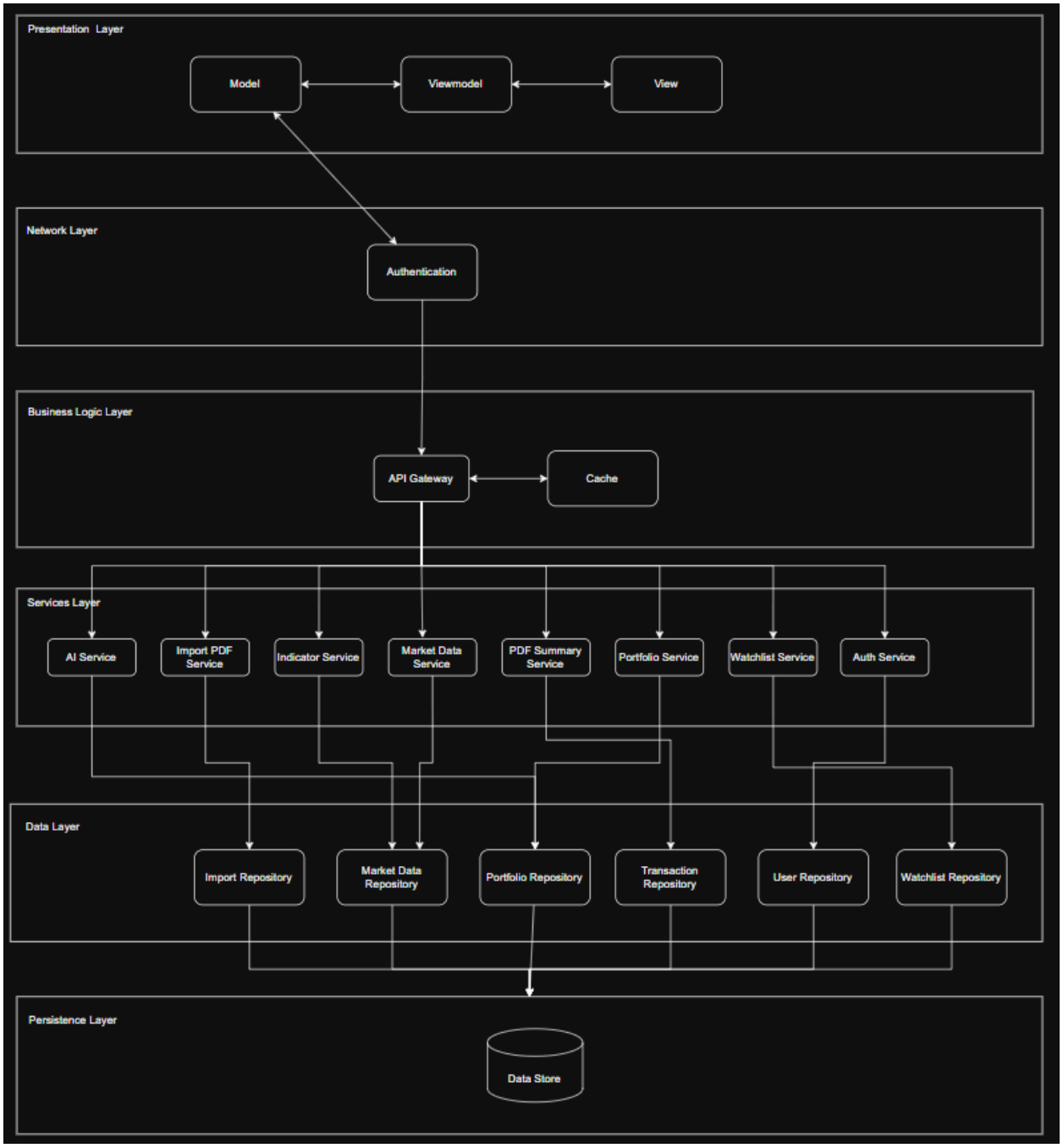
- **C-11:** The conversational AI must include documented guardrails preventing explicit buy/sell recommendations.
- **C-12:** Guardrails must be implemented at the system-prompt level and documented.

2.3.5 Scalability Constraints

- **C-13:** The architecture must support multiple concurrent users and growing volumes of historical market data.

- **C-14:** Horizontal scaling must be achievable without significant architectural changes.

2.4 Architectural Diagram



2.5 Mapping Quality Requirements to Architectural Decisions

Quality Requirement	Justification
Security - 100% of protected endpoints reject unauthenticated requests, MFA enforced, TLS + AES-256, strict per-user data scoping	Authentication delegated to a managed identity provider (AWS Cognito) providing email verification and TOTP MFA, every protected router resolves the user via a shared get_current_user dependency and repositories filter every query by the resolved user's ID, TLS termination at the managed gateway/CDN, managed database and object-store encryption at rest, no brokerage credentials ever collected
Performance – p85 API response <= 1000 milliseconds, dashboard renders <= 5 seconds	Cache-aside pattern for market data: an instrument already in the object-store cache is served with zero provider round-trips, a single aggregated dashboard endpoint (GET /api/portfolio) instead of N per-widget calls, indexed foreign keys on all portfolio-scoped tables, live pricing bounded to what the dashboard displays.
Scalability – The Big Five has no real evidence of this project performing at scale as of yet	Stateless API (session state lives with the client token and database), so the container can be redeployed or replaced without losing session state; relational schema with explicit indexes on all high-volume lookup paths (`portfolio_id`, `ticker`, `user_id`); historical data growth handled by the same indexed query paths.
Usability & Accessibility - first import to populated dashboard <= 5 min, WCAG 2.1 AA, responsive 320px+, reduced-motion support	Single-page application with client-side routing for instant page transitions, design-token-driven theming (light/dark) with contrast-checked variables, global keyboard focus-visible styling, empty states designed as onboarding paths (dashboard prompts import when no holdings exist), prefers-reduced-motion respected by animated components.

3. Technology Requirements

3.1 Frontend

- React + Vite (framework) - component model fits the dashboard's card/panel structure, Vite gives fast dev feedback, large ecosystem and team familiarity reduce delivery risk.
- TailwindCSS + CSS design tokens (styling) - utility classes keep styling co-located with components, CSS variables implement the brand style guide's tokens and light/dark themes.
- Recharts (data visualisation) - declarative React charting for the performance, sector and dividend charts without custom SVG work.
- SheetJS (xlsx) (spreadsheet parsing) - parses the Excel import template client-side before preview/confirm.
- Docker and nginx (hosting) - the production build (Dockerfile.prod) is served by nginx from a container on the same EC2 instance as the backend (Section 5.2).

3.2 Backend

- FastAPI (Python 3.11) (framework) - async-capable, automatic OpenAPI documentation (kept in docs/api/openapi.json), Pydantic validation enforcing the API contracts, Python gives direct access to the financial computation ecosystem.
- pandas + numpy (financial computation) - vectorised indicator calculations over price history.
- SQLAlchemy + Alembic (ORM and migrations) - versioned, reversible schema migrations, CI verifies upgrade to downgrade to upgrade on every run.
- Docker on a single AWS EC2 instance (compute) - the FastAPI app is built into a Docker image and run as a container directly on one EC2 host (docker run -p 8000:8000).

3.3 Data and External Services

- AWS RDS PostgreSQL (database) - relational integrity for the portfolio hierarchy (portfolios to holdings to transactions), encryption at rest, free-tier eligible, indexes satisfy.
- AWS Cognito (authentication) - managed user pool providing email verification codes and TOTP MFA without storing credentials ourselves, tokens verified server-side per request.
- AWS Bedrock (Anthropic Claude) (AI) - managed LLM inference inside the AWS estate, system-prompt guardrails, portfolio context assembled server-side so no third party receives raw user data beyond the inference call.
- yfinance (market data) - free access to JSE price history and fundamentals, covers prices, history and search. Resilience comes from the S3 cache plus the per-holding cost-basis fallback (Reliability) rather than a second provider, adding an independent fallback provider is on the backlog for NFR-3.2 hardening.
- NewsData (news) - free-tier financial news APIs covering South African tickers, named in the SRS deliberately to keep the requirement unambiguous.

3.4 Delivery and Operations

- GitHub Actions (CI/CD) - free for the repository, runs the full quality gate on every push/PR.
- Docker + Docker Compose (containerised development) - one-command reproducible local environment (frontend, backend, database) so a fresh clone of main runs with only the repository's instructions.
- Amazon ECR (container registry) - stores the built frontend and backend Docker images, tagged by commit SHA plus a mutable latest. The deploy workflows pull from here onto the EC2 host.

Development tools: Git (CLI), GitHub (repo, issues, project board), Postman (endpoint testing), VS Code.

Testing frameworks: Vitest + React Testing Library (frontend unit/component), Playwright (E2E), pytest + pytest-cov (backend unit/integration), Codecov (coverage reporting), npm audit / pip-audit / Bandit / Trivy (dependency and security scanning in CI).

4. API Contracts

All endpoints are prefixed /api and return JSON. Unless marked (no auth), endpoints require a bearer access token in the Authorization header and return 401 when it is missing, expired or invalid. Full request/response schemas are auto-generated into docs/api/openapi.json from the backend's validation models.

4.1 Authentication - /api/auth

- **POST** /api/auth/register *(no auth)* - register with full name, email, password. 201 on success, 409 if the email is already registered, 422 on validation failure.
- **POST** /api/auth/confirm *(no auth)* - confirm the 6-digit email verification code. 400 on invalid/expired code.
- **POST** /api/auth/login *(no auth)* - authenticate email + password. Returns either tokens (access_token, id_token, refresh_token) or an MFA challenge (challenge, session) that must be completed. 401 on invalid credentials.
- **POST** /api/auth/mfa/verify-login *(challenge session)* - complete an MFA challenge with the 6-digit TOTP code, returns tokens. 401 on invalid code.
- **POST** /api/auth/mfa/associate - begin TOTP setup, returns the shared secret for QR display.
- **POST** /api/auth/mfa/confirm-setup - verify a TOTP code to activate MFA for the account.
- **GET** /api/auth/me - return the authenticated user's profile.
- **POST** /api/auth/logout - invalidate the user's sessions.

4.2 Portfolio - /api/portfolio

- **GET** /api/portfolio - aggregated dashboard payload: summary (total value, cost, gain/loss, daily change), live-priced holdings, sector allocation, performance history, dividend history. The daily snapshot is recorded on this call.
- **GET** /api/portfolio/summary - summary statistics only.
- **GET** /api/portfolio/sectors - sector allocation breakdown.
- **GET** /api/portfolio/performance - snapshot history (portfolio value + benchmark per day).

4.3 Portfolio Import - /api/import_pdf and /api/import_pdf_summary

Statement content is extracted client-side

- **POST** /api/import_pdf/ - register an uploaded statement document, returns the document record for the save calls below.
- **POST** /api/import_pdf/save_portfolios - persist confirmed portfolio metadata.
- **POST** /api/import_pdf/save_holdings - persist confirmed holdings.
- **POST** /api/import_pdf/save_instrument_purchases_and_sales - persist confirmed buy/sell transactions.
- **POST** /api/import_pdf/save_contributions_and_withdrawals - persist confirmed cash movements.

- **POST** /api/import_pdf/save_dividends_and_withholding_tax - persist confirmed dividend records.
- **POST** /api/import_pdf/save_transaction_expenses - persist confirmed fee/expense records.
- **GET** /api/import_pdf_summary/summary/{portfolioID} - statement summary for an imported portfolio.
- **GET** /api/import_pdf_summary/top_holdings/{portfolioID} - largest holdings in the statement.
- **GET** /api/import_pdf_summary/lowest_holdings/{portfolioID} - smallest holdings in the statement.
- **GET** /api/import_pdf_summary/portfolio_allocation/{portfolioID} - allocation breakdown.
- **GET** /api/import_pdf_summary/trading_activity/{portfolioID} - buy/sell activity.
- **GET** /api/import_pdf_summary/cash_flow/{portfolioID} - contributions and withdrawals.
- **GET** /api/import_pdf_summary/dividend_income/{portfolioID} - dividend income.
- **GET** /api/import_pdf_summary/expenses/{portfolioID} - fees and expenses.

4.4 Market Data - /api/stocks

- **GET** /api/stocks/details?symbol= - current price, volume and daily change for a ticker, error response if no data is available.
- **GET** /api/stocks/history?symbol=&period= - historical OHLCV data, period accepts the standard ranges (1d, 5d, 1mo, 3mo, 6mo, 1y, 2y, 5y, 10y, ytd, max).
- **GET** /api/stocks/search?query= - search stocks by ticker or company name.

4.5 News - /api/news

- **GET** /api/news/all - latest general financial news.
- **GET** /api/news/?category= - news filtered by category, with holding-relevant naming applied server-side.

4.6 AI Assistant - /api/ai_chat

- **POST** /api/ai_chat/ - submit a message ({message, conversation_id?}). Builds portfolio context, applies guardrails, returns {reply, conversation_id}. Starting a conversation auto-generates a title.
- **GET** /api/ai_chat/conversations/ - list the user's conversations (id, title, timestamps), most recent first.
- **GET** /api/ai_chat/conversations/{id}/messages/ - full message history for a conversation.
- **PUT** /api/ai_chat/conversations/{id}/ - rename a conversation ({title}).
- **DELETE** /api/ai_chat/conversations/{id}/ - delete a conversation and its messages.

4.7 Watchlist - /api/watchlist

- **POST** /api/watchlist/ - add a ticker to the user's watchlist ({ticker}), enriched server-side with company name and sector.

- **GET** /api/watchlist/ - list the user's watchlist entries with current market data.
- **DELETE** /api/watchlist/{watchlistID} - remove an entry.

4.8 System

- **GET** /health *(no auth)* - health check, returns {"status": "ok"}. Exposed for external uptime monitoring.

4.9 Financial Indicators - /api/indicators

- **GET** /api/news/all - latest general financial news.
- **GET** /api/news/?category= - news filtered by category, with holding-relevant naming applied server-side.

5. Deployment

5.1 Deployment Requirements

Live, accessible system: the production system is reachable at equitylens.co.za](https://equitylens.co.za/)

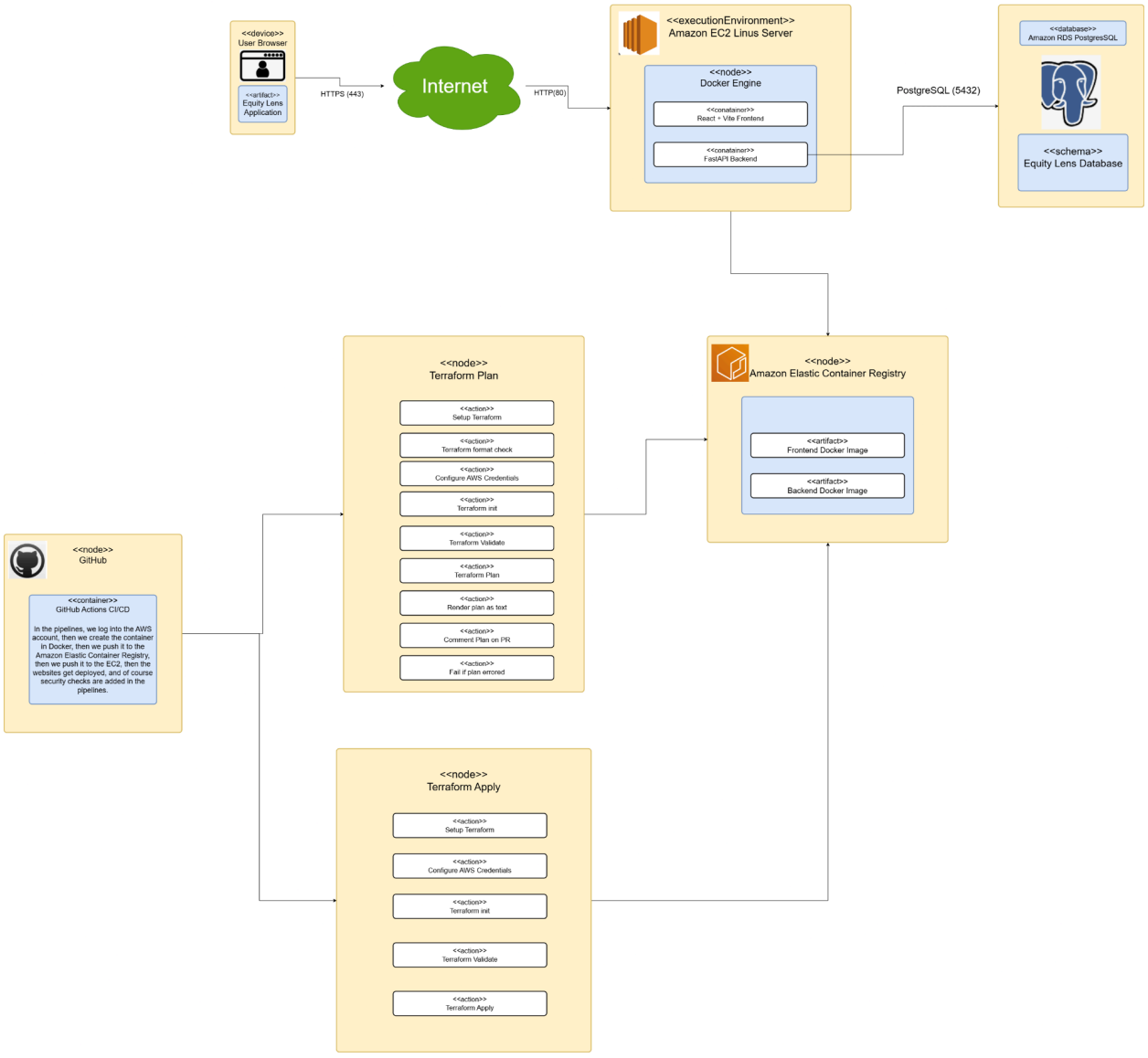
Environment parity: deploy-backend.yml and deploy-frontend.yml both trigger their AWS/deploy steps on pull_request events targeting main or dev, and both deploy to the same fixed container names on the same single EC2 instance. There is no separate staging AWS stack. A PR opened against dev deploys to the same production instance as a PR against main.

Development - Docker Compose on each developer's machine (local PostgreSQL, hot reload)

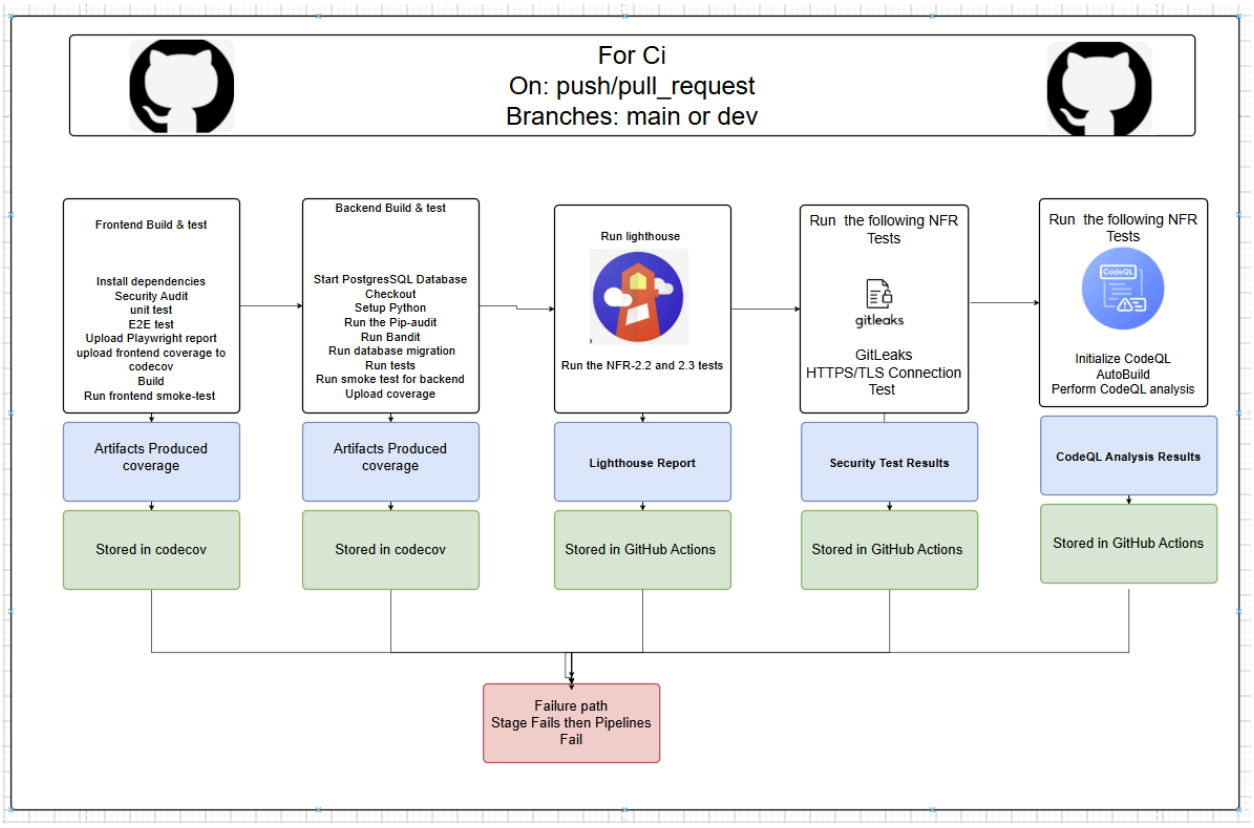
Production - the single EC2 instance, redeployed whenever a qualifying PR event fires

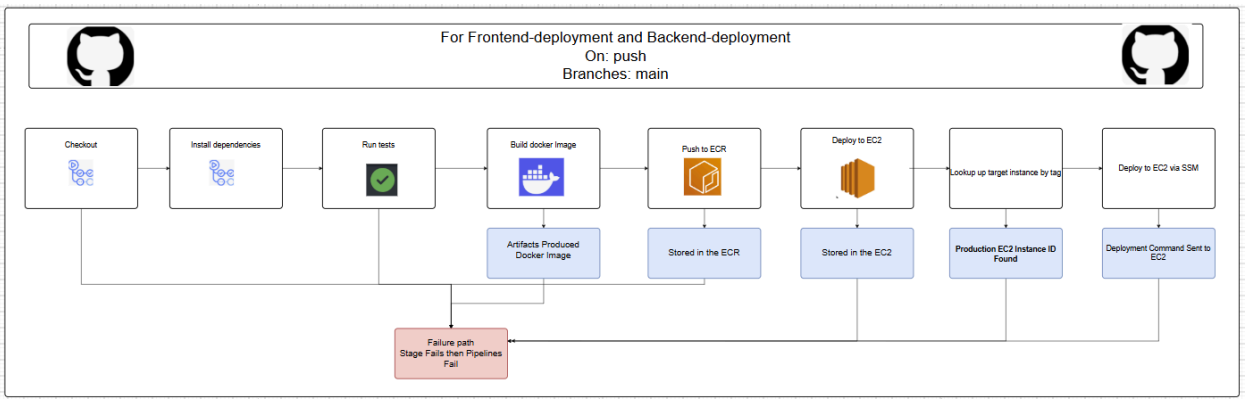
Reproducibility (containerisation) - local environments are fully containerised: a fresh clone of main runs with docker compose up --build using only the repository's instructions. Production is also containerised (Docker images pushed to Amazon ECR, pulled and run on the EC2 host)

5.2 Deployment Diagram



5.3 CI/CD Pipeline Diagram






Terraform Plan
On: Pull Request
Path: infrastructure/terraform/**
Setup
format check
AWS credentials
Run init
Validate
Terraform plan
Render Plan as text
Comment plan on PR
Terraform Apply
On: Manual Trigger
Setup
AWS credentials
Run intt
validate
Apply

5.4 Secrets Management

Sensitive information such as the API keys, and AWS credentials, will be stored secretly in the GitHub Actions (Secrets and environment variables), and we will also ensure that no sensitive information will be committed in any branch for security purposes. To see these variables, you can visit our GitHub in the secrets and environments variables, where you will see the following.

5.5 Rollback Strategy

- Backend: here is no immutable-version rollback anymore. The deploy script always pulls and runs the :latest ECR tag; rolling back today means manually SSHing to the EC2 host and running docker pull <repo>:<previous-sha> followed by the same stop/rm/run sequence the deploy script uses, since ECR does retain each commit's SHA-tagged image. There is no automatic rollback-on-failure: the old container is stopped and removed before the new one starts, so a crash-on-startup in the new image currently means downtime with nothing to fall back to automatically.
- Frontend: The frontend is a Docker/nginx container on the same EC2 instance. Rollback is a manual re-tag-and-redeploy procedure, similar to the one above.
- Database: Alembic migrations are written to be reversible and CI proves it on every run (upgrade to downgrade to upgrade), so a schema rollback is alembic downgrade to the previous revision. RDS automated snapshots cover data-level recovery.
- Process: main only receives merges from dev that passed the full pipeline plus two reviews, so the previous main commit is always a known-good deployable state, the default rollback action is revert the merge commit and let the pipeline redeploy.